

Bombino Express — Final Phase: Module Specification

Companion to the 3-week delivery plan. For each module: **what it is, why it exists, what needs to be built, and how you know it's done**. If a task isn't listed under a module here, it belongs to no one — raise it before building it.

Team: Arbaaz (M-modules) · Aditya (A-modules) **Calendar:** Mon 3 Aug – Sat 22 Aug 2026 · 6-day weeks · 18 working days **Reference:** demo-bombino-express.vercel.app is the visual + UX source of truth

1. What we are building

Bombino Express today is a single-customer booking app: someone at Bombino creates your account, you log in with ITD credentials, you book, and a docket is created immediately.

The final phase turns it into a **self-service courier platform with three connected products in one codebase**:

- **Customer** — signs up themselves, books a door-to-door pickup, pays, and tracks.
- **Pickup Agent** — field staff on a phone, collecting parcels.
- **Operations / Admin** — the internal console that runs every order through its lifecycle.

A demo of all three exists and is approved, but it is a stage set — every API is faked, nothing persists, the role switch is a menu toggle. **The work is building everything behind those screens for real.**

The constraint that shapes everything

ITD dockets cannot be amended after creation. Once a docket and AWB exist, the weight and charges are frozen forever.

So the docket cannot be created at booking, because at booking the weight is only an estimate. Instead:

```
Book → Bombino Order (internal Order ID, no AWB)
      ↓ pickup or drop-off
      At the hub: weigh the real parcel
      ↓
      Reprice, settle the money difference
      ↓
      NOW create the ITD docket, with final values → real AWB
```

This is the **deferred-docket model**, and it means the **Ops Console is the system of record** for every order between booking and dispatch. That's not a feature choice — it's the only arrangement ITD's constraint allows.

2. Module index

#	Module	Owner	Days	Effort	Depends on
M0	Foundation	Both	D1	2.0	—
M1	RBAC & Access Control	Arbaaz	D2½, D11	1.5	M0
M2	Ops Console	Arbaaz	D2½, D3, D5	2.5	M0, M1
M3	Weigh & Settle	Arbaaz	D6–D8	3.0	M2
M4	ITD Integration	Arbaaz	D4	1.0	M0
M5	Docket Finale	Arbaaz	D9	1.0	M3, M4
M6	Status Sync	Arbaaz	D10	1.0	M0
M7	Super Admin (<i>conditional</i>)	Arbaaz	D12	1.0	M1
A1	App Shell & Routing	Aditya	D2	1.0	M0
A2	Identity & Access	Aditya	D3, D4, D6	3.0	M0, M4
A3	Booking	Aditya	D5, D7	2.0	M0
A4	Payments	Aditya	D9	1.0	A3
A5	Pickup Agent App	Aditya	D10, D11	2.0	M0, A1
A6	Customer Views	Aditya	D8, D12	2.0	A3, M6

Only one dependency crosses lanes: A2 → M4. Company signup needs `addCustomer()` . M4 lands D4, A2 needs it D6 — two days of slack. Everything else is same-owner sequencing.

3. M0 — Foundation (*joint, Day 1*)

What it is

The shared vocabulary and machinery both lanes need before either can build anything.

Two developers working in parallel on the same order lifecycle will invent two incompatible mental models unless they agree, once and precisely, on: what an order *is*, what states it can be in, who is allowed to move it between states, and how the client and server talk about all of it. Get that wrong and week 3 becomes a rewrite instead of an integration.

This is one day of coupled work that buys eleven days of complete independence. **It is the only day in the schedule where either developer can block the other.**

The central design idea

Every lifecycle transition — for every role, in every surface — goes through **one endpoint**:

```
POST /api/orders/:id/actions { action: "weigh", payload: { actual_weight: 3.2 } }  
→ { order: Order, availableActions: [ { action, label, requiresPayload } ] }
```

The server computes `availableActions` from the order's current status and the caller's role. **The UI renders a button per entry in that array and never contains the state machine itself.**

That single decision is what decouples the two developers. Arbaaz can add a transition, rename one, or change who's allowed to perform it, and Aditya's agent app picks it up with no code change — and vice versa.

What needs to be built

#	Task	Who
1	Split <code>server/routes.ts</code> (1036 lines) into <code>server/routes/{auth,orders,ops,agent,payments,support,kyc}.ts</code> , self-registering from an index. Verbatim move, zero behaviour change.	Arbaaz
2	Split the client into <code>client/src/pages/{customer,ops,agent}/</code> , each with its own <code>routes.<surface>.tsx</code> mounted in <code>App.tsx</code> . Neither dev edits the other's route file again.	Aditya
3	<code>shared/orderContract.ts</code> — the status vocabulary, <code>Order</code> and <code>Action</code> types, the customer-facing status derivation table, the <code>isPaymentSatisfied()</code> signature. Imported by both server and client.	Joint
4	Agree and write down the table ownership map (§4).	Joint
5	Author and apply the full schema — all five tables, including Aditya's . Nothing deferred to "when we get there."	Arbaaz
6	<code>server/orderLifecycle.ts</code> — the transitions map (one data entry per legal <code>status × action × role</code> triple), the guards, and <code>availableActions(order, role)</code> .	Aditya
7	<code>POST /api/orders/:id/actions</code> — the uniform endpoint above.	Aditya
8	Seed script producing at least one order in every status, shared by both devs as fixtures.	Arbaaz

The frozen status vocabulary

No additions without both developers agreeing.

Pickup path: `pickup_requested` → `agent_accepted` → `out_for_pickup` → `picked_up` → `received_at_hub`
Drop-off path: `awaiting_dropoff` → `received_at_hub`
Both, at hub: `received_at_hub` → `weighed` → `settled` → `ready_for_docket` → `dispatched`
(+ cancelled from most states)

Board columns: **Pickup · At Branch · Settle · Dispatch · Done.**

Definition of done

- [] Seed produces at least one order in every one of the 11 statuses
- [] A role-guarded transition executes end to end, and is rejected for the wrong role
- [] Both lanes compile against `orderContract.ts`
- [] Everything merged to `main` before end of day

Out of scope for Day 1

Reprice logic · notifications · payment rules · anything cosmetic. These belong to modules. **Do not let them leak into Day 1** — the day slipping is the single worst outcome available in this plan.

4. Table ownership

Both developers have SQL access to one shared Supabase project. That removes the last dependency, and introduces a new failure mode: two people running DDL on live tables. Rules:

One DDL owner per table · additive-only for the whole phase · every statement committed as a numbered file in migrations/ · announce before applying · no destructive DDL during working hours.

Additive-only means: no renames, no drops, no type changes. Need a rename? Add the new column, backfill, drop it after the deadline.

Table	DDL owner	Read by	Written by
users	Aditya	All modules	A2 (signup) · M7 (roles)
otp_codes	Aditya	A2	A2
payments	Aditya	M3, A4, A6	A4 (Razorpay, pay-at-pickup) · M3 (pay-at-drop-off)
orders	Column-partitioned	All	A3 booking columns · M3/M5 fulfilment columns
order_events	Arbaaz	M2, A6	All modules — insert-only, so no conflict
shipments , tracking_events (existing)	Arbaaz	A6	M5
kyc_documents , addresses , notifications , audit_log (existing)	Arbaaz	—	A2 · A3 · M6

orders is the one table both lanes write, so it's partitioned by column:

Aditya owns (booking)	Arbaaz owns (fulfilment)
order_no , user_id , pickup_request , pickup_date , pickup_slot , consignee , origin_address_id , items , booked_weight , quoted_amount , payment_method , payment_status , is_cod	status , agent_id , actual_weight , final_amount , awb_no , itd_docket_response

metadata JSONB exists on orders and payments as an escape hatch — stash a field mid-task without a migration, promote it to a real column later.

5. Arbaaz's modules

M1 — RBAC & Access Control · 1.5 days · D2½, D11

What it is

Today the app has one kind of user. The final phase has three, with genuinely different powers: a customer must not be able to weigh a parcel or generate a docket; an agent must not be able to reach the settle screen; ops staff must be able to do both.

The demo faked this with a role toggle in a menu — anyone could become an admin by picking it from a dropdown. This module makes it real. **Role lives on the user record, and every request is checked on the server.** A customer who types an ops URL, or curls an ops API directly, gets a 403 — not a page with the buttons hidden.

The second half of this module (D11) is a deliberate hardening pass late in the schedule, after every route exists, going route by route with three sessions in hand and confirming each one rejects who it should.

What needs to be built

1. `role` column on the user record, loaded into the session at login
2. `requireRole(...roles)` middleware, sitting alongside the existing `requireUser`
3. Apply the guard to every ops and agent route as they're written
4. A consistent 403 JSON shape the frontend can render sensibly
5. Config endpoints the other surfaces need: hub list, pickup slot definitions
6. **D11 hardening sweep** — enumerate every route, test with customer/agent/admin sessions, fix every gap
7. Confirm agents self-accept jobs — no admin-assignment path is built anywhere

Endpoints	<code>GET /api/config/hubs</code> · <code>GET /api/config/slots</code>
Tables	<code>users</code> (read <code>role</code>)
Out of scope	Super Admin user management (M7) · hiding UI per role — each surface owner does their own, and it's cosmetic only

Definition of done

- [] Customer session → 403 on **every** ops and agent route, verified route by route
- [] Agent session → 403 on ops-only routes, 200 on agent routes
- [] Admin session → 200 on ops routes
- [] 403s return a consistent shape the frontend renders as a real message
- [] No admin-assignment path exists for pickups — agents self-accept

M2 — Ops Console · 2.5 days · D2½, D3, D5

What it is

The internal surface where Bombino staff run the business.

Because dockets are deferred, **this console is the system of record for every order between booking and dispatch**. An order's entire pre-transit life — waiting for pickup, out with an agent, sitting at the hub, being weighed, being settled — happens here and nowhere else. If this screen is wrong, the business is wrong.

Staff open it and see every live order sorted into the phase it's in. They click one, a drawer opens with the full detail, and they act on it. Desktop gets a kanban board because staff work on laptops at the hub; mobile gets a clean grouped list because the same staff check it on a phone.

Old shipments from before this phase still exist in the database — they bucket into a separate group, collapsed by default, so they don't clutter the live board.

The design idea that keeps this small

The action area of the drawer is **generic**. It reads `availableActions` from the API response and renders one button per entry, with a small form for actions that need input (weigh needs a number, collect payment needs an amount). It contains no knowledge of the lifecycle.

This is why 2.5 days is realistic for a board plus a detail view built from scratch. Without it, this module is four days or more, and every new transition means touching UI code.

What needs to be built

1. Board query — fetch live orders, grouped into the five lifecycle phases
2. Desktop kanban layout, five columns
3. Mobile grouped-list layout with identical grouping
4. A generic `OrderCard` — order no, customer, route, weight, amount, payment state, age
5. Legacy/other bucket, collapsed by default
6. Order detail drawer — full order, address, items, payment state, event history
7. Generic action-sheet renderer driven by `availableActions`, with per-action-type input forms

Screens	<code>/ops</code> board · order detail drawer
Endpoints	<code>GET /api/ops/orders?phase=</code> · <code>GET /api/ops/orders/:id</code>
Tables	Reads <code>orders</code> , <code>order_events</code> , <code>users</code>
Out of scope	Weigh/settle logic and its UI (M3) · docket action (M5) · status derivation (M6)

Definition of done

- [] All 11 statuses render in the correct column
- [] The drawer shows exactly the legal actions for the caller's role — nothing hardcoded
- [] Mobile grouped list matches desktop grouping
- [] Legacy bucket present and collapsed by default
- [] Board reflects another user's action after a refresh

M3 — Weigh & Settle · 3.0 days · D6–D8

What it is

The commercial heart of the whole phase, and the reason the deferred-docket model exists.

A customer books at an **estimated** weight and pays, or promises to pay, an estimated amount. The parcel then arrives at the hub and weighs something else — usually more. Because the ITD docket freezes charges permanently the moment it's created, that difference has to be found and settled **before** the docket exists. There is no fixing it afterwards.

So at the hub, staff put the parcel on a scale and type in the real weight. The system reprices, compares against what was booked, and shows the difference in plain money terms: **collect ₹340 more**, **refund ₹120 due**, or **no change**. The order physically cannot advance toward a docket until the weight is captured and the payment situation is resolved.

Two of the four payment methods are collected by hand, and one of them is collected here — **pay at drop-off**, where the customer brings the parcel to the hub and pays staff on the spot. That collection UI lives in this module because it lives in this surface.

COD is the exception to everything: it's never collected by Bombino at all. It's a flag for the accounts team, and it must **not** block the docket.

What needs to be built

1. `weigh` action — capture actual weight against the order
2. Reprice service — compute the final amount from the actual weight
3. Delta computation — classify as `collect_more` / `refund_due` / `no_change` with the amount

4. Weight entry UI in the ops drawer
5. Delta banner UI making the outcome unmissable
6. `collect_payment` action + pay-at-drop-off collection UI
7. Reconciliation against advance payments — an advance-paid order that weighs up owes a balance
8. `isPaymentSatisfied(order)` — the single helper both lanes use
9. `settle` action with guards: no settle without weight, no settle without payment satisfied (COD excepted)

Actions	<code>weigh</code> · <code>collect_payment</code> · <code>settle</code>
Tables	<code>orders</code> (write <code>actual_weight</code> , <code>final_amount</code> , <code>status</code>) · <code>payments</code> (write) · <code>order_events</code>
Out of scope	Razorpay refund execution — manual flag this phase · COD collection (never collected by us) · Razorpay integration itself (A4)
Decision needed D6	Reprice source: fresh ITD rate call at the weighed weight, or a stored rate card? Recommend fresh call, booked quote as fallback

Definition of done

- [] Book 2 kg, weigh 3 kg → `collect_more` with the correct amount
- [] Book 2 kg, weigh 1 kg → `refund_due` with the correct amount
- [] Book 2 kg, weigh 2 kg → `no_change`
- [] `settle` rejects an unpaid non-COD order with a clear, renderable error
- [] COD order settles freely and stays flagged for accounts
- [] No order reaches `ready_for_docket` without weight captured **and** payment satisfied

M4 — ITD Integration · 1.0 day · D4

What it is

The single place in the codebase that knows how to talk to ITD.

`server/itd.ts` already handles token fetching, rate calculation, tracking, and docket creation. This module adds **corporate customer creation** — when a company signs up, a matching customer record has to be created inside ITD via `add_customer` — and pulls every ITD-specific constant out of the code into config.

The config part matters more than it sounds: three ITD values are still unconfirmed by Anas, and the whole point is that **they must not block anything**. If they're constants in an env file, they drop in on the day they arrive and nothing needs rebuilding.

There's a wrinkle worth knowing before starting: ITD uses **two different auth schemes**. `docket_api` takes a session token from `get_token`. `customer_api` takes a static Bearer. They are not interchangeable.

This module also owns resolving the docket attribution question (§7) — the one genuinely unresolved piece of architecture in the plan.

What needs to be built

1. `itdClient.addCustomer()` — form-urlencoded POST, static Bearer auth, typed payload
2. `server/itdConfig.ts` — every ITD constant read from env, zero literals in code
3. The 24-hub table (name-code-id) as a typed constant

4. Verify `show_kyc_after_login` actually persists — the sample response echoes it back mangled, which looks like an ITD parsing bug
5. Resolve docket attribution with Anas and define the customer-identity parameter M5 will use

Confirmed config (from `CUSTOMER_CREATION_API_DETAILS.txt`):

- `company_id = 2` — production `.env` already uses it; the sample's `1` is ITD's own test company
- `location_code` is a **3-letter code** (`MUM`), separate from hub, and **not** cross-validated against `hub_id`
- `customer_setting = {"show_kyc_after_login":1,"is_fsc_apply":1}`

Tables	None
Out of scope	The <code>create_docket</code> call itself (M5) — though the customer-identity parameter shape is decided here
Blocks	A2 company signup (D6) — the schedule's only cross-lane dependency

Definition of done

- [] `addCustomer()` creates a real ITD customer against sandbox and returns whatever identity ITD provides
- [] Hub table exported as a typed constant, 24 entries
- [] `hub_id`, `company_id`, `location_code`, `bearer` all read from `env`
- [] `show_kyc_after_login` confirmed to have actually persisted on the ITD side
- [] Docket attribution answered, with the parameter shape written down for M5

M5 — Docket Finale · 1.0 day · D9

What it is

The irreversible moment. Everything before it is Bombino-internal and editable; everything after it is frozen in ITD forever.

This module takes an order that has been weighed and settled and converts it into a **real ITD shipment with a real AWB**. It runs exactly once per order, at the very end, using the final confirmed weight and amount — because ITD will never permit an amendment.

From this point the Order ID maps to an AWB, the customer's order becomes trackable, and the existing tracking machinery (already built and working) takes over.

It's a one-day module because most of the machinery exists: `itdClient.createShipment()` works, and `persistShipmentAfterCreate()` already knows how to write a `shipments` row and its addresses. The work is **rerouting** that path — moving it out of booking and into the end of the ops lifecycle, fed by settled values instead of estimates.

What needs to be built

1. `generate_docket` action, available only on a `ready_for_docket` order to an admin
2. Build `CreateShipmentPayload` from **final settled values**, not booked estimates
3. A hard guard against double-firing — a second call must be impossible, not merely unlikely
4. Call `createShipment()` with whatever customer identity attribution resolved to
5. Persist `awb_no` and the full `itd_docket_response` on the order
6. Create the `shipments` row via the existing `persistShipmentAfterCreate` path
7. Ready-for-docket state UI and the AWB reveal

Tables	orders (write awb_no , itd_docket_response , status) · shipments
Out of scope	The customer-facing post-AWB tracking view (A6)
Blocked by	create_docket payload confirmation from Anas — known open issue on freight amount / api_service_code / kg-vs-lb

Definition of done

- [] A settled order produces a real sandbox AWB
- [] Order flips to `dispatched` ; the `shipments` row exists
- [] Existing tracking works against the new AWB
- [] Firing the action twice on one order is impossible
- [] The values sent are final weight and final amount — never the booked estimate

M6 — Status Sync · 1.0 day · D10

What it is

The customer never sees internal vocabulary. They don't know what `ready_for_docket` means and they shouldn't have to.

This module is the **translation layer plus the fan-out**. Every time ops or an agent moves an order, three things happen: the customer's view of that order updates to a human phrase, a notification row is written, and an audit row records who did what and when.

The subtle requirement is **silence**. Three internal transitions — weigh, settle, ready-for-docket — must produce *no* visible customer change at all. The parcel is at the hub; from the customer's point of view nothing has happened yet. Firing a notification for each of those would be noise.

Because all of this is derived and persisted server-side, state is genuinely shared: two staff members and the customer looking at the same order see the same thing, and a refresh shows current truth rather than stale browser memory.

What needs to be built

1. `deriveCustomerStatus(order)` in `orderContract.ts` — internal status → customer-facing phrase
2. Notification insert on every customer-visible transition
3. Audit log row on **every** transition, visible or not
4. Guarantee internal-only transitions produce no notification and no customer-visible change

The mapping, from the brief:

Internal	Customer sees
agent_accepted	Pickup confirmed
out_for_pickup	Agent on the way
picked_up	Parcel picked up
received_at_hub	Arrived at Bombino hub
weighed / settled / ready_for_docket	<i>(nothing — internal)</i>
dispatched	In transit (+ <i>becomes trackable</i>)

Tables	notifications , audit_log , order_events
Out of scope	The customer UI that displays any of it (A6)

Definition of done

- [] All six mappings produce the right customer-facing label
- [] Every transition writes exactly one `order_events` row and one `audit_log` row
- [] Internal-only transitions produce no notification
- [] State survives refresh and is identical across two browsers

M7 — Super Admin · 1.0 day · D12 · conditional

What it is

A tier above ops that manages internal users — creating agent and admin accounts, assigning roles, deactivating people who leave.

Scope is unconfirmed. The brief lists it as desirable, not required, and notes it may be handled manually for now. Confirm with Bombino by D12. If it's out, this day becomes ops console polish and buffer — which, given the estimates elsewhere, is not a bad outcome.

What needs to be built (*if in scope*)

1. Internal user list
2. Create internal user with a role
3. Change role / deactivate
4. Guard the whole surface to `super_admin` only

6. Aditya's modules

A1 — App Shell & Routing · 1.0 day · D2

What it is

One codebase now serves three different products. A customer, a pickup agent, and an ops staffer open the same URL and must land in what feel like completely different applications — different navigation, different home screen, different everything.

This module builds that skeleton **before any of the screens inside it exist**: the routing trees, the role-based landing and redirect logic, and the per-surface shells. It's deliberately first, because every other A-module needs somewhere to render.

It also produces the **fixtures** — orders in every lifecycle status — so that UI work can proceed at full speed on realistic data. These should match Arbaaz's seed script so both developers are looking at the same world.

One thing to be clear about: the guarding here is **cosmetic**. It stops a customer stumbling into the ops UI. It is not security — M1 is the security, enforced server-side.

What needs to be built

- 1. Route trees: customer at / , ops at /ops/* , agent at /agent/*
- 2. Role-aware guard — redirect anyone who doesn't belong on a surface
- 3. Role-aware landing — each role lands on its own home
- 4. Per-surface navigation shells
- 5. Fixtures covering all 11 statuses, matched to the seed data

Out of scope	The screens themselves (A2–A6, M2) · real access control (M1)
--------------	---

Definition of done

- [] Customer navigating to /ops or /agent is redirected
- [] Agent sees agent nav only; admin sees ops nav
- [] Fixtures render every status
- [] Each role lands on the correct home screen after login

A2 — Identity & Access · 3.0 days · D3, D4, D6

What it is

Today, someone at Bombino creates your account by hand and you log in with ITD credentials. This module replaces that with **genuine self-service signup**.

A person opens the app, enters their phone number, receives a real SMS code, chooses **Personal** or **Company**, fills in their details, and is logged in and booking within a couple of minutes. No one at Bombino touches anything.

The two paths diverge meaningfully:

- **Personal** — collect name, email, phone, address, plus Aadhaar (number and an optional document upload). Every personal customer rides a **single shared retail ITD account** underneath, but each is a distinct Bombino user on our side, individuated by our own records.
- **Company** — collect company details plus GST number, and on completion **create a real ITD customer** via `add_customer` .

Aadhaar and GST are **format-validated only**. No live verification API this phase — that's explicitly excluded and would be a paid add-on.

The other important behaviour: **KYC is captured once here and never asked again**. The old flow re-prompted during booking. It doesn't anymore, for either account type.

What needs to be built

1. OTP provider integration (real SMS)
2. `POST /api/auth/otp/request` and `/verify`, with `otp_codes` storage and rate limiting
3. Account chooser screen — Personal or Company
4. Phone entry + OTP entry screens, reusing the existing `input-otp` component
5. Personal path: details form + Aadhaar capture, reusing the existing `kycUpload` component and `kycDb`
6. Company path: company details + GST field, format-validated
7. Call `itdClient.addCustomer()` on company completion, store the returned identity
8. Issue the session and land the user in the booking flow

Screens	Chooser · phone entry · OTP entry · personal details · Aadhaar · company details + GST
Endpoints	<code>/api/auth/otp/request</code> · <code>/verify</code> · <code>/api/auth/signup/personal</code> · <code>/api/auth/signup/company</code>
Tables	<code>users</code> , <code>otp_codes</code> (DDL owner) · <code>kyc_documents</code> (existing)
Out of scope	Paid Aadhaar verification API · paid GST registry API · assigning any role other than <code>customer</code> · re-asking KYC at booking
Blocked by	OTP provider + credentials (D3) · shared retail ITD credentials (D4) · M4 for company path (D6)

Definition of done

- [] Both paths end with the user logged in and a `users` row created
- [] Personal signup writes a `kyc_documents` row
- [] Company signup stores `itd_customer_id`
- [] Real SMS arrives; a wrong code is rejected; attempts are rate-limited
- [] Booking never asks for KYC again, for either account type

A3 — Booking · 2.0 days · D5, D7

What it is

The booking flow already exists and is polished — 2,800 lines of working `CreateShipment.tsx` with rate calculation, address picking, HSN codes and country data. **This module extends it rather than rebuilding it.**

Two things change.

First, additions to the form: the customer now chooses whether Bombino picks the parcel up or they drop it off. If pickup, they choose a date and one of four three-hour windows between 9 AM and 9 PM. They also choose how they want to pay, from four options.

Second — and this is the important one — what happens when they submit. Today, submitting creates an ITD docket and returns an AWB. **That must stop.** Submitting now creates a Bombino order with an internal Order ID and returns that instead. The AWB comes days later, from ops.

That means the existing docket-on-booking path has to be genuinely rerouted, not merely bypassed. Booking must fire **zero** ITD requests.

The order created here has to carry everything ops needs downstream, because ops has no other source: consignee, origin/pickup address, real amount, pickup choice and slot, payment method, and booked weight.

What needs to be built

- 1. Pickup vs drop-off toggle, mapping to ITD's pickup_request (1 = pickup, 2 = drop-off)
- 2. Date picker plus four 3-hour slot options, 9 AM–9 PM
- 3. Payment method selector — the four methods
- 4. POST /api/orders — persist the order, no ITD call
- 5. Order number generator, BOM-XXXXXX
- 6. Write the first order_events row
- 7. Reuse the existing findOrCreateAddress for address dedup
- 8. Remove the docket call from the booking path — this is a deletion, and it needs to be deliberate
- 9. Rework the success screen to show the Order ID, not an AWB

Screens	Extended CreateShipment.tsx · Order ID success screen
Endpoints	POST /api/orders · GET /api/orders
Tables	orders (booking columns), order_events , addresses
Out of scope	Payment execution (A4) · any ITD call whatsoever

Definition of done

- [] Booking fires zero ITD calls — verified in the request log
- [] Success screen shows an Order ID and no AWB anywhere
- [] The order carries consignee, origin address, real amount, pickup choice + slot, payment method, booked weight
- [] Order persists and survives a refresh

A4 — Payments · 1.0 day · D9

What it is

Money is entirely Bombino's problem. ITD handles none of it — no payment field is ever sent to ITD.

The customer picks one of four methods at booking:

Method	When collected	By whom	Built in
Pay now (advance)	At booking	Razorpay	A4
Pay at pickup	At the doorstep	Pickup agent marks collected	A5
Pay at drop-off	At the hub	Admin marks collected	M3
Pay at delivery (COD)	At destination	Not collected by us — flag only	A4

This module builds the gateway integration for the one method that's collected up front, plus the COD flag. The two physically-collected methods are built in the surfaces where the collection actually happens — the agent's phone and the ops console — because that's where the button belongs.

Two rules that matter downstream. Because the amount is an estimate until the parcel is weighed, an advance payment does not end the story — it still reconciles at settle, and M3 handles the collect-more or refund. And COD must never block the docket; it settles and dockets freely, just flagged for accounts.

Refunds this phase are a **manual flag**, not a gateway refund — decision due D7.

What needs to be built

1. Razorpay order creation endpoint
2. Pay-now checkout UI
3. Signature verification
4. Webhook handler, **idempotent** — replaying it must not double-credit
5. Write `payments` rows and maintain `orders.payment_status`
6. COD flag at booking, with no payment row created

Endpoints	POST <code>/api/payments/razorpay/order</code> · <code>/verify</code> · <code>webhook</code>
Tables	<code>payments</code> (DDL owner) · <code>orders.payment_status</code>
Out of scope	Pay-at-drop-off collection (M3) · pay-at-pickup collection (A5) · refund execution — manual flag this phase
Blocked by	Razorpay confirmation + keys (D9)

Definition of done

- [] Test-mode payment completes and writes a `payments` row
- [] Webhook is idempotent — replaying it does not double-credit
- [] COD order books with the flag set and no payment row
- [] **COD does not block the docket**
- [] No payment field is sent to ITD anywhere

A5 — Pickup Agent App · 2.0 days · D10, D11

What it is

A field tool for the person on a bike.

They open the app and see pickups nobody has taken yet. They tap one to claim it — **agents self-accept, there's no dispatcher assigning jobs** — and it moves to their own list. Then they work through a short sequence of buttons as they do the job: heading out, arrived and collected, and if the customer chose pay-at-pickup, taking the money at the door.

When they drop the parcel at the hub, the job **leaves their list entirely** and ops takes over. That handoff is clean and one-directional.

Design constraints that come from the job, not from taste: it must work **one-handed on a phone**, possibly on bad network, by someone standing on a street. Big targets, short flows, obvious next action.

The one genuinely tricky engineering problem is **claiming**. Two agents can open the available list at the same moment and tap the same job. Exactly one must win, and the other must get a clean, comprehensible rejection rather than a silent failure or a double-assigned parcel.

Explicitly **not** in this phase: any live map or GPS tracking. The brief excludes it. Status updates only.

What needs to be built

1. Available pickups list

2. **Race-safe claim** — concurrent accepts resolve to exactly one winner
3. My pickups list
4. Field action screens: out for pickup → picked up → received at hub
5. Pay-at-pickup collection, writing a `payments` row
6. Handoff — the order disappears from the agent queue at `received_at_hub`
7. Mobile-first layout throughout

Screens	<code>/agent/available</code> · <code>/agent/mine</code> · field action screens
Endpoints	<code>GET /api/agent/pickups/available</code> · <code>/mine</code> · transitions via the uniform action endpoint
Tables	Reads <code>orders</code> · writes <code>order_events</code> , <code>orders.agent_id</code> , <code>payments</code>
Out of scope	Live map / GPS tracking (explicitly excluded this phase) · admin assignment of jobs

Definition of done

- [] Two agents accepting the same job concurrently → one wins, the other gets a clean rejection
- [] Order leaves the agent queue at `received_at_hub` and ops takes over
- [] Pay-at-pickup collection writes a `payments` row
- [] Usable one-handed on a phone — this is a field tool, not a dashboard

A6 — Customer Views · 2.0 days · D8, D12

What it is

What the customer sees after they've booked.

The complication is that in this model **an order has two lives**. Before the docket exists, it's a Bombino order with an internal Order ID and internal progress — no AWB exists anywhere, and showing one would be a lie. After the docket, it's a real trackable ITD shipment with an AWB and live tracking events.

This module makes **one set of screens handle both**, switching mode when the docket lands. The existing `Orders.tsx` and `ShipmentDetails.tsx` become dual-mode rather than being replaced.

The timeline is built from `order_events`, rendered through the customer-facing labels M6 derives — so the customer sees "Agent on the way", never `out_for_pickup`. And because three internal transitions are deliberately silent, the timeline sits still while the parcel is being weighed and settled at the hub. That's correct, not a bug.

The last requirement is that **it must be current**. An action taken in the ops console has to show up here without the customer doing anything clever, and a hard refresh has to show real state rather than stale local storage.

What needs to be built

1. Order list, dual-mode — orders and shipments in one list
2. Order detail, dual-mode — pre-AWB and post-AWB layouts
3. Lifecycle timeline built from `order_events` using derived customer labels
4. Notifications screen wiring (`Notifications.tsx` already exists)
5. Polling / refetch so status changes appear without a manual reload
6. Post-AWB handoff into the existing tracking timeline
7. Mobile polish

Screens	Orders.tsx , ShipmentDetails.tsx (extended), Notifications.tsx (wired)
Tables	Reads orders , order_events , notifications , shipments , tracking_events
Out of scope	Status derivation itself (M6) · push notification infrastructure

Definition of done

- [] A pre-AWB order renders with no AWB shown anywhere
- [] An ops action becomes visible to the customer within one poll interval
- [] Hard refresh shows current truth, not stale local state
- [] After docket, the view switches to the existing tracking timeline
- [] Internal-only transitions produce no visible change

7. Open architectural question — docket attribution

Resolve before D4. This is the one genuinely unresolved piece of architecture in the plan.

`create_docket` has **no customer field** — see `server/itd.ts:294` , `createShipment(data, token)` . Attribution is a function of *whose session token* makes the call. That works today because customers log into ITD with their own credentials and book for themselves.

The deferred-docket model breaks that assumption: **ops** creates the docket, days after booking, from a staff account. For the docket to land on the right corporate ITD account, ops needs that customer's token — but `add_customer` **creates no credentials and returns no customer id**. Its response only echoes the request back.

Three resolutions, in preference order — for Anas:

1. **`create_docket` accepts `customer_code` / `customer_id`** alongside the Bombino company token. Most likely — `getRates` already takes a `customer_code` parameter (`itd.ts:271`). If this works, corporate customers never authenticate to ITD at all and A2 gets simpler.
2. **`add_customer` provisions credentials** that we store per customer. The machinery already exists: `server/crypto.ts` has `encryptPassword` / `decryptPassword` , and `itd_users` already stores per-user ITD tokens. Costs A2 roughly half a day.
3. **All dockets ride the Bombino company account**, with customer attribution kept Bombino-side only. Simplest, but loses per-corporate billing inside ITD.

Interim design so this blocks nothing: store `itd_customer_id` as nullable, and have the docket call take customer identity as a **parameter**. Any of the three resolutions drops in without a redesign.

8. Blockers

Item	Needed by	Chase
Docket attribution (§7)	D4	Arbaaz → Anas
OTP/SMS provider + credentials	D3	Aditya → Bombino
Production Bearer for <code>customer_api</code>	D4	Arbaaz → Anas
Shared retail ITD account credentials	D4	Arbaaz → Anas
<code>hub_id</code> + <code>location_code</code> assignment rule	D4	Arbaaz → Bombino
Refund mechanics — manual flag vs gateway refund	D7	Aditya → Bombino
Razorpay confirmation + keys	D9	Aditya → Bombino
<code>create_docket</code> payload (freight / <code>api_service_code</code> / kg-vs-lb)	D9	Arbaaz → Anas
Super Admin in scope?	D12	Arbaaz → Bombino

Already resolved: `company_id = 2 · location_code semantics · hub id/code table · full add_customer` payload shape.

9. Explicitly out of scope — do not build

Live map / real-time GPS pickup tracking · Voice AI agent · Live/paid Aadhaar KYC verification API · Live/paid GST registry API · Master tracking / US-airport segregation.

Aadhaar and GST are **format-validated only**. If any of these come up, flag to Arbaaz — they are future paid phases, not this one.

10. If the schedule slips

Cut order: M7 Super Admin → M2 ops desktop polish → A5 agent visual refinement. All three are trailing-edge; none blocks another module.

Week 3 (D13–18, 12 man-days) carries **no module work** — it is entirely integration and hardening. That third of the schedule is what makes the module estimates survivable. The moment feature work leaks into it, the estimates lose the slack behind them.